

*Scrape ~100 real Polish listings → deterministic + AI normalization → MySQL → filterable Vue frontend with natural-language chat search. The approach was decided up front; the build followed it.*

## 1 WHAT DATA I EXTRACT, AND WHY

The fields people actually filter and decide on: **price, area, rooms, floor, city/district, offer type (sale/rent), description, images, source URL** — plus two honesty-driven fields: **administrative rent (czynsz)**, which materially changes true monthly cost and appears on sale offers too, and a **completeness flag** so incomplete offers stay visible rather than silently dropped. I store the **raw source payload** beside each normalized row so every stage is reproducible offline, and skip low-value extras (coordinates, galleries, price history). The ~100 are a **stratified random draw** across the whole country from random page offsets (never the ad-boosted top of list), balanced at 54 sale + 54 rent so both sides of the filter have real depth.

## 2 HANDLING UNSTRUCTURED / LOW-QUALITY DATA

The governing principle is **null beats wrong**. A missing or unparseable value is stored as explicit `null` and rendered as `"—"` — never guessed, never defaulted to `0` or `"Unknown"`. A user who sees `"—"` knows the value is unknown; a user who sees a fabricated price decides on a lie. **Flag, never drop**: incomplete offers are badged and kept — the messiness is exactly what the task probes. **Duplicates: flag, never delete** — an idempotent upsert on a natural key means re-running ingestion never duplicates, and a fuzzy pass flags re-posts for inspection.

## 3 WHERE AND WHY I USE AI

The pipeline is **deterministic-first**: regex / structured-field parsing handles everything code can read (price, area, rooms, floor, location) at **zero token cost**. The **LLM is invoked only on the residue** — prose-only gap-fills and factual summaries — constrained to extract **only what the text explicitly states** (null otherwise). This cuts per-listing token spend and shrinks the hallucination surface; numeric fields never reach the model. Chat search maps a vague query to the **same** structured filters the manual UI produces — one implementation, two input modalities. Every touchpoint degrades gracefully: with no API key the app still browses, filters, and searches.

## 4 ONE KEY ASSUMPTION

A single marketplace's **structured fields are more reliable than free-text prose**, so I trust the source's typed values and use AI only to fill what the structure lacks. One snapshot of ~100 offers is enough to demonstrate the full ingest → normalize → serve → search pipeline; scale is explicitly not the goal.

## 5 ONE SUCCESS METRIC

A vague chat query returns  $\geq 2$  **contextually relevant listings with zero manual filters**, with the counter-metric **zero fabricated listings** (every result exists in the DB with a real source URL). Confirmed on committed data: *"a cheap flat in Warszawa"* → { city: "Warszawa", maxPrice: 650000 } → 3 results; *"flat around 40 m<sup>2</sup> under 500000"* → 12.

## 6 ONE FAILURE MODE & LIMITATION

The intent parser can **hallucinate a price ceiling** for a vague word like "cheap". Mitigation: interpreted filters are echoed in a **visible banner and land in the normal filter controls**, so the user sees exactly what was applied and can clear or adjust it — nothing happens invisibly. Broader limitation: the balanced sale/rent split does not mirror the real ~86/14 market composition — an intentional trade for a ~100-row demo.

## 7 WHAT I'D IMPROVE WITH MORE TIME

A **second marketplace** as a source (parsers written portal-tolerant, so it's an adapter not a rewrite); **search at scale** via FULLTEXT / a vector index for semantic chat beyond  $n \approx 100$ ; **API/E2E coverage** beyond the unit-tested normalization core; and **ranking** that surfaces explicit matches above unknown-value rows.